

微瑕非遗产品交易平台—测试计划

团队编号：第 9 组

项目成员：2251310 杜天乐 2251916 罗诗雨 2252443 吴婉宁 2252444 赵思源

1 项目范围

1.1 项目背景

“微瑕非遗产品交易平台”是一个融合电商交易、社区互动与非物质文化遗产推广于一体的复合型应用系统。项目围绕“瑕疵非遗产品”的市场缺口与文化价值进行创新设计，旨在为传统非遗商品打造新的流通与展示路径，拓宽其受众面和销售渠道。

传统非遗手工制品往往因轻微瑕疵而被淘汰，无法进入主流市场，造成资源浪费与文化表达受限。平台通过引入“微瑕甄选”机制，赋予商品新的价值标签，鼓励消费者接受文化传承中的不完美之美，增强文化共鸣与环保意识。

本系统面向三类角色提供服务：

- 普通买家**：浏览商品、下单交易、收藏评价、参与社区互动；
- 商家用户**：提交资质、发布商品、管理订单、参与市集与营销；
- 平台管理员**：内容与用户审核、举报处理、主题市集运营与数据监控。

平台主要功能包括但不限于：

- 用户注册登录与权限控制
- 商家资质认证与审核机制
- 商品上架、库存维护、图文展示、订单管理
- 支持积分抵扣、余额支付、支付宝在线支付等多种结算方式
- 社区模块支持发帖、评论、点赞、举报与消息推送
- 后台系统支持审核流程、舆情监管、数据统计与权限封禁

平台采用 ASP.NET Core MVC 框架，配合 Vue.js 前端组件库，基于 Oracle 数据库与 Entity Framework Core 实现数据持久化操作，同时接入第三方服务如支付宝 SDK、SMTP 邮件认证系统与分布式 ID 生成组件。

该平台为一个**全功能上线项目**，具有复杂的业务流程、高并发交互需求及多种角色权限体系，其测试工作量大、覆盖面广、风险点密集。构建高效、系统、完整的测试计划是保障系统稳定、安全、高质量交付的关键。

1.2 总体目标

1.2.1 推动微瑕非遗商品价值重构

本平台旨在为存在细微缺陷的非遗手工制品重新赋予商业与文化价值，鼓励消费者以包容的态度接纳不完美的美学形式，提升社会对非遗传承与手工精神的认同感。

平台通过“微瑕甄选”“限量销售”等机制，打造特色商品标签体系，让传统产品在新时代语境中获得新的生命力。

1.2.2 构建完整电商交易链条

项目通过系统化构建商家入驻、商品上架、购物下单、支付结算、评价反馈、售后维权等全流程模块，形成支持多种支付方式（积分、余额、支付宝）、多类型用户身份（商家、买家、管理员）、多状态流转（待付款、待发货、维权中等）的标准电商流程体系。

系统目标是：实现功能完整、流程闭环、操作流畅、数据可追踪的非遗电商生态。

1.2.3 融合社区与互动机制提升用户黏性

项目不仅聚焦交易，还集成社区互动功能，允许用户通过发帖、评论、收藏、举报等形式参与到平台生态中，增强使用者之间的连接，提升用户的参与感与文化认同感。

社区功能与商品系统打通，可实现“从文化了解到商品购买”的一站式体验，满足现代用户内容驱动型购物习惯。

1.2.4 实现系统的可运营性与可扩展性

平台采用前后端分离架构（Vue.js + ASP.NET Core + Oracle），提供良好的可维护性与拓展性，为后续引入新模块（如直播带货、非遗课程）、对接小程序或 APP 版本、拓展多语种支持等提供良好基础。

在系统层面，平台将实现：

- 高并发支持：满足市集、秒杀等流量高峰场景
- 权限精细化控制：保障各角色间的数据隔离与功能访问边界
- 数据可视化与运营支撑：为平台管理方提供商家数据、用户活跃、交易流水等核心运营数据

2 测试对象

2.1 主要功能特性

本平台的核心业务流程包括用户注册登录、商品交易、支付结算、社区互动与后台管理，具体功能模块如下：

模块	功能编号	功能名称	涉及角色	简要说明
用户管理	F-01	注册与登录	所有角色	支持邮箱验证登录、角色识别与权限分配
用户管理	F-02	身份认证与权限控制	所有角色	按用户类型分配权限与访问范围
商品交易	F-03	商品浏览与搜索	买家	支持分类筛选、关键词检索与商品收藏
商品交易	F-04	商品发布与库存维护	商家	支持商品图文上传、定价、库存设置等
订单系统	F-05	下单、支付与订单管理	买家、商家	包含订单生成、支付扣款、发货确认等流程
支付系统	F-06	支付宝在线支付、积分抵扣	买家	可选择余额支付、积分兑换与支付宝接口

模块	功能编号	功能名称	涉及角色	简要说明
钱包系统	F-07	充值、提现与积分明细	买家、商家	支持资金流水与积分兑换逻辑
社区系统	F-08	帖子发布、评论、点赞举报	所有角色	构建平台用户互动与非遗文化传播社区
通知系统	F-09	消息推送与已读状态管理	所有角色	支持多设备同步与消息分类管理
管理系统	F-10	用户与商品审核、举报处理	管理员	实现内容监管、违规处理与权限封禁

2.2 非功能特性

本平台还需满足以下关键的非功能需求，以保障整体系统的质量、可维护性与安全性。

类别	编号	非功能特性	说明
性能	NF-01	支持高并发访问	登录、秒杀、支付等模块支持 1000QPS
安全	NF-02	数据加密传输与访问控制	敏感信息加密、基于角色的访问权限控制
可用性	NF-03	异常处理与容错机制	网络异常、支付失败等具备应急回滚能力
可维护性	NF-04	支持模块化扩展与配置中心	各功能模块解耦，支持热更新与灰度发布
可监控性	NF-05	实时系统监控与告警	使用 Prometheus + Grafana 进行监控与分析
国际化	NF-06	接口支持多语言切换	支持中英文界面及数据字段语言适配
合规性	NF-07	支持政策配置与法律合规	提供地区税收、退货规则等策略切换能力

2.3 软件架构及主要组件

本平台采用典型的 **前后端分离 + 分层式架构设计**，后端基于 ASP.NET Core MVC 框架，前端使用 Vue.js + Element UI，数据库为 Oracle，采用 Entity Framework Core 实现 ORM 映射。整体组件划分如下：

层级	主要技术	说明
前端展示层	Vue.js + Element UI	用户界面交互，组件化开发，适配多端访问
后端服务层	ASP.NET Core MVC	提供 RESTful API 接口，处理业务逻辑与权限校验
数据持久层	Oracle + Entity Framework Core	支持 Code First 模式，复杂业务表结构映射
中间件	Nginx + Cookie 登录验证	网关转发、Session 保持、负载均衡
外部接口	支付宝 SDK、SMTP 邮箱服务	实现支付、验证码、第三方登录等关键功能支持
工具组件	Yitter.IdGenerator	雪花算法实现分布式 ID 唯一生成

层级	主要技术	说明
管理后台	管理员专用模块	商品与用户审核、举报处理、数据统计与权限管理

平台采用现代 Web 三层架构 + 模块化业务设计，确保系统具备良好的扩展性、可维护性与部署灵活性。测试对象覆盖以下关键技术组件与架构设计层次：

表示层（前端）：

- 使用 **Vue.js + Element UI** 实现响应式页面构建；
- 前端通过 **Axios** 调用后端 **RESTful** 接口；
- 实现路由控制、身份校验、错误提示与界面交互逻辑；
- 所有用户操作由前端统一封装，便于测试路径构造与自动化脚本执行。

业务逻辑层（后端）：

- 基于 **ASP.NET Core MVC** 框架构建，采用控制器 + 服务类设计；
- 所有功能接口通过 **Web API** 暴露，支持跨域访问与身份鉴权；
- 集成中间件组件实现请求日志、异常处理、Token 校验等通用逻辑；
- 支持配置中心与环境变量注入，便于环境迁移和灰度控制。

数据访问层：

- 使用 **Entity Framework Core** 实现 ORM 映射，支持 Code First 与自动迁移；
- 操作底层 **Oracle 数据库**，业务数据模型映射至三十余张关系型表结构；
- 所有关键交易流程使用事务控制与数据一致性断言；
- 外键约束、唯一索引、数据校验规则嵌入模型，防止异常数据写入。

外部依赖组件：

组件	功能说明
支付宝支付 SDK	实现订单支付请求发起、支付回调处理、验签校验等功能
SMTP 邮件服务	用户注册、密码找回等流程中发送验证码邮件
Cookie 鉴权机制	登录后生成加密 Cookie 实现会话保持
Yitter 雪花算法	实现全平台唯一的主键 ID 分发，支持分布式环境

2.4 角色测试覆盖视角

本测试将覆盖以下三类用户角色的业务流程与权限边界：

- **普通用户（买家）**：商品浏览、下单支付、社区互动、积分使用、售后处理等核心操作流程；
- **商家用户**：商品上架、资质审核、订单发货、营销活动参与、查看数据报表等；
- **管理员角色**：后台管理控制台、内容审核、用户封禁、市集配置、数据监控等系统权限操作。

3 高层次测试套件设计

本章节根据系统模块划分测试套件，覆盖组件级、集成级、系统级和验收级各测试层级，确保平台核心业务的功能正确性、交互一致性及关键风险点的有效防控。各测试套件根据业务重要性与潜在风险进行优先级标注，作为后续测试实施与资源配置的重要依据。

3.1 用户与身份管理测试套件（TS-01）

测试目标：
确保用户注册、登录、身份识别与账户管理等功能在不同输入场景下行为稳定、安全合规，防止账号滥用、撞库攻击与伪造身份等潜在威胁，提供清晰的交互流程与异常提示。

测试覆盖范围与测试层级：

测试层级	主要测试对象	测试点细化	涉及风险点	方法与工具
组件级	- 邮箱、手机号正则匹配逻辑 - 验证码生成与格式限制 - 密码强度检测	- 邮箱格式验证边界值（包含中文、非法字符） - 验证码有效期/频率机制逻辑 - 密码加密是否使用加盐算法（如BCrypt）	- 验证码爆破绕过 - 密码暴露/弱密码通过	xUnit + 正则单测 + 安全加密测试（F12抓包验证传输）
集成级	- 注册接口 ↔ 用户表 ↔ 验证服务调用 - 登录接口 ↔ 鉴权模块 ↔ Cookie 设置	- 新用户注册后信息是否写入数据库 - 同一邮箱多次注册拦截逻辑 - 登录成功是否生成会话 Cookie 并含有效标识	- 注册失败无响应 - Cookie 丢失导致越权访问	Postman 接口调用 + Mock 断网/错误返回测试
系统级	- 注册→登录→完善信息→实名认证完整链路 - 多设备并行登录状态一致性	- 同时在 Web 和移动端登录后消息同步验证 - 邮箱验证点击链接跳转逻辑完整 - 实名认证失败后的提示与处理流程<	- 登录异常跳转 - 用户信息未落库/越权行为	Selenium + 本地代理模拟环境切换 + 数据比对脚本
验收级	- 新用户首次注册的引导流程完整性 - 错误输入场景（重复邮箱、格式错误、密码不一致）下的反馈是否清晰 - 验证码错误三次后是否锁定机制生效	用户在首次操作过程中的感知流畅度与安全提示明显性	- 流程中断影响转化率 - 提示信息模糊影响信任感	手动走查 + 可用性记录表 + 异常路径走查表

补充说明：

- 安全测试重点验证 CSRF Token 是否存在、是否可被篡改
- 可引入验证码发放频率上限测试（如5次/小时）
- 测试异常输入组合如手机号为空 + 验证码失效是否被系统识别

测试优先级： 高（High）

覆盖率目标建议：

- 注册与登录接口路径覆盖率 ≥ 100%
- 异常验证类用例 ≥ 20 条，错误码覆盖 ≥ 90%
- 前端交互性逻辑测试覆盖 ≥ 80%

3.2 商品与订单管理测试套件（TS-02）

测试目标：

验证商品发布、展示、购物车、下单、订单生成、发货与用户评价等交易主流程的准确性与稳定性，确保商品信息完整、库存状态准确、订单金额精确计算、订单状态生命周期清晰，避免因商品数据异常或流程中断导致交易失败或用户投诉。

覆盖范围与测试层级：

测试层级	主要测试对象	关键测试点	涉及风险点	测试方法与工具
组件级	商品属性校验逻辑、购物车模型、订单对象构造、金额计算方法	- 商品标题/描述/价格是否为空校验 - 多规格商品 SKU 参数是否匹配准确 - 金额计算是否支持满减+积分叠加精度控制	商品属性缺失导致页面错误价格浮点计算误差	单元测试（xUnit, Pytest）+ Decimal精度校验
集成级	商品服务 ↔ 库存服务 ↔ 订单服务 ↔ 用户服务	- 商品上下架是否影响搜索展示结果 - 下单时是否实时锁定库存 - 订单状态是否与用户钱包扣款同步	库存未锁定造成超卖订单状态与支付状态不一致	接口联调自动化测试 + Mock模拟用户下单场景
系统级	浏览 → 加购 → 提交订单 → 支付 → 评价全过程	- 全路径验证前后端联动与状态同步 - 异常场景处理如支付失败、取消订单等 - 评论与订单绑定是否正确	订单卡住不出库、评论错乱	Selenium 自动化 + 并发下系统稳定性测试

测试层级	主要测试对象	关键测试点	涉及风险点	测试方法与工具
验收级	用户购买流程与异常提示完整性	- 商品缺货、网络失败、支付失败是否有明确提示 - 下单后取消与退款流程是否顺畅 - 用户可否查询订单详情、物流信息	用户操作混乱、信任下降	手动测试 + 多终端交互走查 + 异常流程手动回归

补充验证点：

- 校验“商品已下架但用户购物车仍可下单”的场景是否有效拦截
- 检查前端商品详情页中的图片加载、属性标签、标签展示是否与数据库一致
- 模拟用户同时下单限量商品，验证系统是否触发并发库存控制（使用悲观/乐观锁）
- 评论上传图片是否做大小/格式/非法内容限制

测试优先级： 关键（Critical）

覆盖率目标建议：

- 商品/订单主流程覆盖率 ≥ 95%
- 金额计算测试场景 ≥ 10 类（含积分叠加、满减、优惠券）
- 异常订单状态模拟 ≥ 5 类（如取消、超时、支付失败、售后）
- 评论功能测试路径 ≥ 100%（含匿名、有图、追评）

3.3 支付与钱包系统测试套件（TS-03）

测试目标：

确保用户钱包充值、余额支付、积分抵扣、退款退回、支付回调、账单明细等交易闭环关键路径准确无误，保证所有金额精度处理、状态流转、交易日志保持一致，避免资金损失与对账异常。

覆盖范围与测试层级：

测试层级	主要测试对象	关键测试点	涉及风险点	测试方法与工具
组件级	金额计算模块、积分使用逻辑、钱包余额扣除逻辑	- 金额精度：Decimal(18,2)处理是否一致 - 积分优先使用策略与余额支付逻辑是否互斥- 订单退款时余额/积分退回路径是否一致	金额丢分位、积分重复抵扣、退款失败	单元测试 + 浮点误差测试 + 钱包余额断点回归测试

测试层级	主要测试对象	关键测试点	涉及风险点	测试方法与工具
集成级	钱包 ↔ 订单 ↔ 支付宝 ↔ 积分系统	- 订单金额与支付金额是否一致 - 第三方支付回调状态是否精准更新订单 - 交易失败时是否发起退款并生成对应记录	回调丢失或异常重复回调、订单状态不同步	模拟支付回调接口 + 并发支付回调测试 + Mock支付宝响应
系统级	用户充值 → 下单支付 → 成功支付 → 退款 → 钱包更新	- 核心路径中断时能否回滚余额和积分 - 是否支持异步回调延迟处理 - 支付异常场景下用户状态恢复是否准确	钱未到账但状态已完成、重复支付等	UI操作脚本 + 事务一致性测试 + Mock超时模拟
验收级	钱包账单明细页面、支付安全提示、余额变更提醒	- 每一笔交易记录是否有唯一 ID、清晰备注与金额明细 - 支付失败是否提示重试、安全验证是否到位 - 钱包界面余额展示与实际一致	用户不信任平台账目、误扣款	黑盒测试 + 审计日志手工核对 + 模拟账单出错场景

补充验证点：

- 模拟支付超时、支付宝返回无签名、签名错误、重复回调，系统是否具备幂等性控制机制
- 检查交易状态表与日志表是否同步更新（如状态为“失败”是否仍生成扣款记录）
- 测试积分/余额不足下的提示清晰度与引导性
- 测试退款路径是否按原支付方式返回（余额/积分/支付宝）

测试优先级： 关键（Critical）

覆盖率目标建议：

- 所有支付路径场景测试 ≥ 12 类（含正常/失败/回调/退款）
- 回调接口用例覆盖 ≥ 100%，幂等控制测试 ≥ 5 条
- 钱包明细字段验证 + 状态一致性 ≥ 95%
- 所有失败路径都应返回明确错误码和提示信息

3.4 商家入驻与管理测试套件（TS-04）

测试目标：

验证商家注册、资质审核、商品发布、订单处理、收益结算、售后服务等全流程是否合规，确保平台与商家之间的数据同步、权限控制与结算机制运行稳定，避免假商户、审核绕过、违规商品发布或漏结算问题。

覆盖范围与测试层级：

测试层级	主要测试对象	关键测试点	涉及风险点	测试方法与工具
组件级	商家注册表单、证件上传组件、审核信息模型	- 提交内容是否按格式校验（如手机号、营业执照号） - 上传图片格式/大小限制是否正确处理 - 商家编号与注册信息唯一性校验	非法商户通过审核、信息缺失、商户数据错配	表单验证单元测试 + 模拟上传测试（Mock图片）
集成级	商家服务 ↔ 审核服务 ↔ 商品服务 ↔ 后台系统	- 审核通过是否及时更新商家状态为可发布 - 审核失败是否阻止商品上架权限 - 商家与管理员通知是否联动	审核结果与权限不一致、商品误放行	接口集成测试 + 审核接口状态差异比对脚本
系统级	入驻 → 审核 → 商品发布 → 接单 → 结算 → 售后	- 跨多模块完整链路数据一致性与流程跳转测试 - 商家违规是否可被管理员封禁、商品撤回 - 订单收入结算、周期性汇总逻辑准确性验证	商品违规未拦截、误封商家、结算数据缺漏	E2E 场景驱动测试 + 流程事务一致性校验
验收级	商家后台 UI、导航结构、数据反馈	- 操作逻辑是否清晰、功能入口是否可发现 - 错误提示是否可理解，操作反馈是否及时 - 商家数据统计、销售额、售后率是否清晰呈现	操作复杂、功能错位、反馈延迟	可用性走查 + 商户用户访谈反馈 + 任务完成时长记录

补充验证点：

- 审核失败后修改资料是否能触发重新审核流程
- 检查多角色登录下商户是否能访问买家模块
- 测试商家删除商品后其订单是否保留历史记录
- 模拟商家高频修改价格、库存，验证系统限制与防刷策略

测试优先级：高（High）

覆盖率目标建议：

- 商家注册与审核流程路径 ≥ 95%
- 发布权限与状态联动验证用例 ≥ 10 条
- 商家后台各模块 UI 可访问性测试 ≥ 100%

- 多角色权限隔离用例 ≥ 100%

3.5 社区互动模块测试套件（TS-05）

测试目标：

验证社区模块中发帖、评论、点赞、举报、内容审核、封禁操作等交互流程的合法性与安全性，确保系统在开放用户内容上传的前提下，具备良好的内容治理机制和舆情管理能力，预防违规内容扩散与攻击行为。

覆盖范围与测试层级：

测试层级	主要测试对象	关键测试点	涉及风险点	测试方法与工具
组件级	发帖表单组件、评论限制逻辑、敏感词过滤接口	- 帖子标题/正文字符长度限制校验 - XSS/SQL 注入拦截机制验证 - 敏感词识别覆盖率与误判率控制	内容绕过审核、输入非法代码、平台形象受损	安全单测（OWASP Top10）+ 正则敏感词黑名单测试
集成级	内容发布系统 ↔ 用户系统 ↔ 举报系统 ↔ 审核模块	- 评论是否绑定用户身份 - 举报是否成功生成后台任务并更新状态 - 举报处理后是否更新举报方通知	举报未生效、处理延迟、误伤正常内容	Postman API 自动化 + Mock 审核系统模拟
系统级	发帖 → 评论 → 点赞 → 举报 → 审核 → 反馈 → 封禁流程	- 模拟用户违规行为被举报、审核通过后用户状态是否更新 - 同一内容多次举报合并机制是否生效 - 违规用户是否能绕过限制发帖	举报冤假错判、审核不透明、封禁失败	回归测试 + SLA 计时模拟 + 后台联动数据校验
验收级	社区交互页面交互性、提示准确度、系统反馈流畅性	- 多设备评论点赞是否同步显示 - 删除帖子是否及时反馈、评论是否同步清除 - 举报反馈是否明确“通过/驳回”并附理由	流程中断、提示混乱、用户体验差	手工走查 + 可用性调研问卷 + 社区热点测试模拟

补充验证点：

- 敏感词更新后是否实时生效（缓存机制验证）
- 举报未处理超 24h 是否触发预警或后台自动提醒机制
- 模拟 IP 段刷评论、刷赞，检查系统拦截策略与阈值设定
- 测试话题热度排序机制是否存在作弊空间

测试优先级： 中 (Medium)

覆盖率目标建议：

- 社区主流程（发帖→评论→举报）路径覆盖 ≥ 95%
- 敏感词命中率测试用例 ≥ 30 条，误报率控制 ≤ 5%
- 举报处理机制 SLA 测试 ≥ 2 类（及时/超时）
- 内容展示同步性验证 ≥ 100%

3.6 平台运营后台测试套件 (TS-06)

测试目标：

确管理理员用户在后台管理系统中能够高效执行平台治理相关任务，包括用户管理、举报处理、商家审核、商品控制、内容封禁、数据可视化、系统配置等操作，平台权限分级明确、操作有据可查，在高并发访问或批量操作场景下系统依然稳定、安全可控。

覆盖范围与测试层级：

测试层级	主要测试对象	关键测试点	涉及风险点	测试方法与工具
组件级	权限判断模块、导航菜单配置组件	- 管理员、普通用户、商家之间的功能访问差异控制 - 左侧菜单跳转 URL 与实际组件绑定逻辑是否一致	权限越权、跳转异常、页面加载失败	RBAC权限映射测试 + Selenium导航测试
集成级	登录认证 ↔ 权限模块 ↔ 日志记录系统	- 登录成功后角色判断是否一致 - 每个操作行为是否生成唯一日志记录条目 - 操作行为是否写入审计表	操作无记录、行为难追踪	Postman接口联调 + 日志Mock比对脚本
系统级	商家审核 → 商品封禁 → 举报处理 → 用户封禁 → 图表刷新	- 跨模块联动逻辑是否完整，通知是否联动（商家、用户） - 数据看板数据是否真实反映业务数据 - 批量审核/封禁是否影响性能	审核错判、误禁言、数据不同步	后台流程自动化测试 + 并发模拟封禁操作
验收级	后台数据清晰度、交互体验、批量操作与回退机制	- 操作按钮布局合理性、二次确认机制 - 批量封禁/审核后若误操作是否支持恢复 - 错误提示是否准确指向出错项	操作难用、误操作风险高、提示不清晰	可用性测试 + 管理员用户访谈 + 功能误操作模拟

补充验证点：

- 审核功能测试：是否支持多管理员同时操作同一审核队列并避免冲突
- 检查操作权限配置表中是否存在未授权接口调用可能性
- 检查所有敏感操作（如封禁、解除）是否提供完整操作日志和变更历史记录
- 可视化图表中不同日期维度数据是否和数据库实际统计一致

测试优先级： 高 (High)

覆盖率目标建议：

- 核心模块页面访问功能路径覆盖率 ≥ 100%
- 审核/封禁流程回归路径 ≥ 10 种
- 所有操作生成日志覆盖率 = 100%
- 后台操作角色权限互斥冲突率 = 0%

3.7 非功能性需求测试套件 (TS-07)

测试目标：

对平台整体在性能 (Performance)、安全性 (Security)、稳定性 (Stability)、兼容性 (Compatibility)、易用性 (Usability) 五个非功能性维度展开专项测试，保障平台在高并发、复杂交易、大规模用户访问场景下的持续可用、安全可靠、响应及时、体验良好。

覆盖范围与测试层级：

测试维度	主要测试对象	关键测试点	涉及风险点	测试方法与工具
性能测试	首页接口、下单接口、支付接口、搜索接口	- 每个接口响应时间 < 1 秒，峰值 TPS ≥ 200 - 并发 1000 用户同时下单是否卡顿 - 活动期间秒杀流量峰值是否崩溃	接口超时、服务宕机、数据写入失败	JMeter、Locust、Redis缓存压测脚本
安全测试	登录、注册、支付、权限接口	- SQL 注入、XSS、CSRF、Token 伪造检测 - Cookie 是否开启 HttpOnly + Secure - 是否存在接口权限越权漏洞	数据泄露、用户信息被窃、账户被盗	OWASP ZAP、BurpSuite、Postman脚本注入测试

测试维度	主要测试对象	关键测试点	涉及风险点	测试方法与工具
稳定性测试	长时运行系统稳定性、线程池、资源池	- 模拟系统运行48小时，监控内存占用是否稳定 - 是否出现资源泄漏、GC频繁或线程死锁 - 页面是否会累积响应时间拉长	服务崩溃、内存溢出、请求阻塞	持续压测 + Prometheus + Grafana 监控曲线分析
兼容性测试	Chrome、Firefox、Safari、Edge、移动端H5	- 页面样式是否错乱，按钮是否响应正常 - 不同平台字体、图片、滑动交互一致性	样式错乱、功能不可用	BrowserStack、SauceLabs、多端手工走查
易用性测试	表单校验、错误提示、用户导航、交互布局	- 错误提示语言是否通俗、是否就近展示 - 页面布局逻辑是否便于理解与操作 - 导航路径是否符合用户使用习惯	用户迷失路径、跳转频繁、投诉增加	用户测试任务 + 访谈反馈 + A/B测试方案对比

补充验证点：

- 站点全路径 HTTPS 是否启用，证书是否有效
- 用户访问失败页面是否显示“友好错误页面”而非系统异常栈
- 首页响应时间稳定性：5分钟一刷新接口耗时浮动是否小于 200ms
- 模拟多个用户连续点击或刷新导致接口限频策略触发验证
- 所有错误码返回是否标准化（如：400/403/500）并附提示

测试优先级： 关键（Critical）

覆盖率目标建议：

- 性能压测覆盖所有核心交易接口（≥10个）
- OWASP Top10 风险点测试 ≥ 100%
- 跨终端页面一致性验证 ≥ 98%
- 友好提示覆盖所有典型错误类型 ≥ 100%

4 测试时间安排

4.1 测试级别及目标

为了确保系统的功能完整性、稳定性和用户体验，测试活动将分为四个主要层级，每个层级具有明确的测试目标和覆盖范围：

测试级别	主要目标	时间周期示例
组件级	验证各模块内部逻辑和功能 correctness	项目初期，单元测试阶段
集成级	检查模块间接口和数据流的正确性与完整性	组件完成后紧接进行
系统级	模拟真实业务流程，验证系统整体功能和性能表现	集成测试完成后执行
验收级	验证系统是否满足用户需求及业务场景的适用性	系统测试后期，与用户验收阶段并行

通过分层测试，逐步覆盖从代码单元到整体系统的功能，保证早期发现问题并及时修正，提高测试效率和质量。

4.2 详细测试时间安排

测试套件	测试层级组合	时间区间	持续时间	说明
用户与身份管理 (TS-01)	组件+集成	5.20 - 5.22	3天	合并基础测试与接口验证
	系统+验收	5.23 - 5.24	2天	用户流程模拟与体验验证
商品与订单管理 (TS-02)	组件+集成	5.20 - 5.23	4天	跨模块数据流重要，略加延长
	系统+验收	5.24 - 5.25	2天	真实业务流快速演练验收
支付与钱包管理 (TS-03)	组件+集成	5.21 - 5.24	4天	与订单并行进行
	系统+验收	5.25 - 5.26	2天	安全流程模拟重点保证
商家入驻与管理 (TS-04)	组件+集成	5.24 - 5.26	3天	并行于支付系统测试
	系统+验收	5.27 - 5.28	2天	后台与商家操作流程压缩验证
社区内容与互动 (TS-05)	全流程并行	5.25 - 5.28	4天	以体验为导向的一体化快速测试
平台运营后台 (TS-06)	全流程并行	5.27 - 5.30	4天	后台多模块联动集中测试

测试套件	测试层级组合	时间区间	持续时间	说明
非功能性测试 (TS-07)	全流程 (性能+安全)	5.31 - 6.07	8天	可并行于功能性测试结尾阶段

4.3 测试时间安排说明

本次测试计划总周期为 **2025年5月20日至2025年6月7日，共19天**。为确保在有限时间内高效推进测试任务，测试安排综合考虑了模块依赖关系、缺陷修复机制与资源调度需求，具体说明如下：

4.3.1 测试顺序与阶段依赖

测试按如下顺序推进，确保质量可控、逻辑清晰：

- 组件级测试优先开展**：各模块单元功能须验证通过后方可进行联调，保障模块基本稳定。
- 集成级测试在组件验证完成后启动**，重点验证模块之间的数据流与接口联动。
- 系统级测试在主要集成完成后执行**，模拟真实业务流程，验证系统整体行为、性能与异常处理能力。
- 验收级测试集中安排在测试后期（6月初）**，由用户代表参与，验证系统是否满足业务需求及用户体验标准。

为适应压缩周期，部分模块采取**并行测试与阶段交叉推进**的方式，确保时间利用最大化，同时保证各模块内部遵循合理顺序。

4.3.2 缺陷修复与回归测试安排

测试过程中发现的缺陷将按优先级及时反馈开发团队，开发修复完成后需安排回归测试，验证修复效果并排查潜在回归问题。

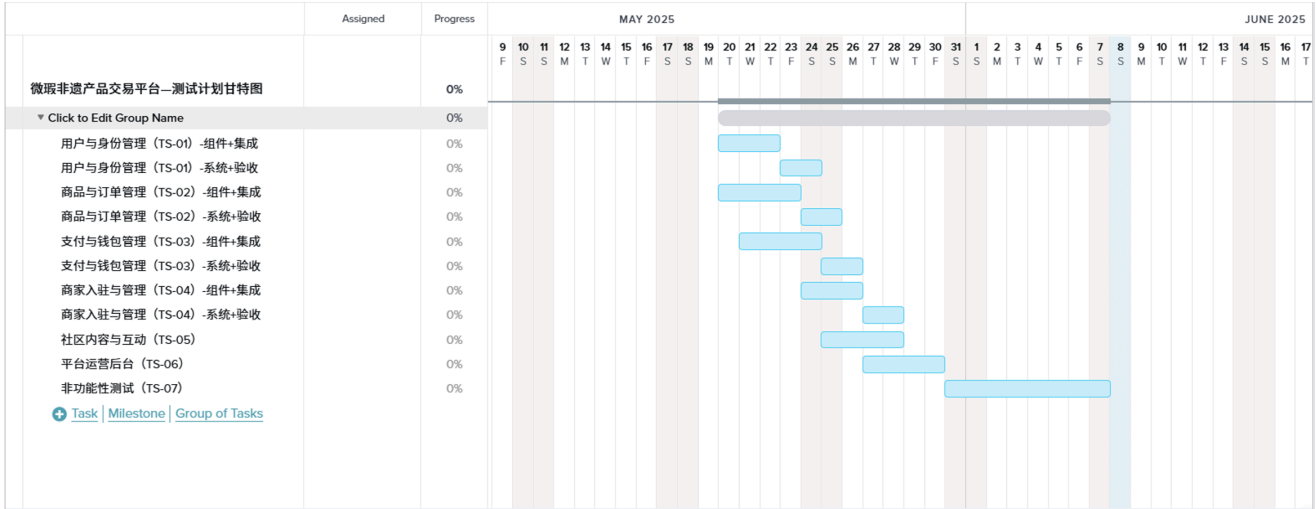
- 回归测试一般安排在 **集成级与系统级测试之间**，必要时亦可在验收前进行快速复验；
- 计划中**6月5日至6月7日**设为**缺陷修复与回归测试缓冲期**，可根据实际情况灵活调度。

4.3.3 资源协调与风险缓冲

考虑到测试期间可能存在人员安排冲突、环境不稳定、接口延迟等突发情况，整体安排中预留约 **10%的时间作为风险缓冲**。主要体现在：

- 测试任务交叉区保留1天流动窗口，便于灵活调整；
- 测试尾期 **6月6日至6月7日**为**通用缓冲窗口**，可用于缺陷修复、补测、验收准备或文档整理；
- 测试人员按模块分组，资源错峰使用，确保测试流程连续高效。

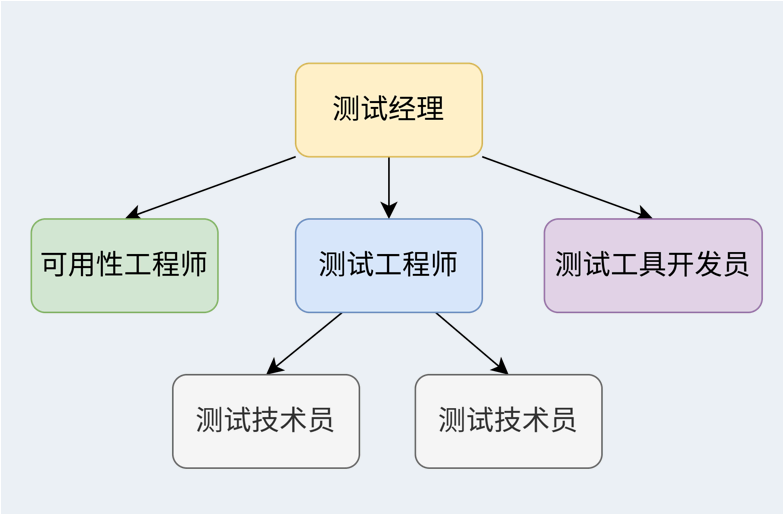
4.4 甘特图



5 组织结构图

职位名称	人数建议 (6人小团队)	职责描述
Test Manager (测试经理)	1人	- 负责测试团队的整体管理与方向制定 - 制定测试策略、测试计划和质量标准 - 分配资源, 协调项目进度和风险 - 作为测试团队与开发/产品/管理层的接口 - 推动测试流程改进、培训与绩效评估
Usability Engineer (可用性工程师)	1人	- 关注产品的用户体验、交互逻辑与易用性 - 设计可用性测试 (如认知走查、A/B测试、用户访谈) - 分析用户行为数据, 提出产品改进建议 - 与产品经理、UI设计师紧密合作 - 在测试中加入UX评估标准
Test Engineers (测试工程师)	1人	- 根据需求文档与设计文档设计测试用例 - 执行手工测试, 提交并跟踪缺陷 - 编写测试报告, 参与用例评审与冒烟测试 - 回归测试、边界测试、异常场景测试等 - 也可根据技能承担部分接口测试
Test Toolsmiths (测试工具开发员)	1人	- 负责开发和维护自动化测试脚本/框架 - 构建测试工具, 如Mock服务、数据生成器、CI集成脚本 - 与开发协作提升系统可测性 - 参与持续集成、自动部署测试的优化流程

职位名称	人数建议（6人小团队）	职责描述
Test Technicians（测试技术员）	2人	- 负责测试环境搭建、配置与维护 - 执行简单重复的测试任务（如冒烟、安装包验证等） - 整理测试数据，记录测试日志 - 支持其他测试人员的环境/工具问题 - 保持测试设备、账户、工具等可用性



6 测试框架选择及原因

为保障本平台的功能完整性、接口稳定性、性能表现与用户体验，测试团队综合项目技术架构、开发语言、测试目标与人员技术栈，制定如下测试框架组合：

6.1 自动化测试框架

类型	工具/框架名称	适用范围	选择理由
接口测试	Apifox + Newman	后端 API 接口正确性与稳定性	支持 RESTful 接口调试、断言检查、自动化回归测试集成
UI 测试	Selenium + Python	前端界面交互自动化测试	跨浏览器支持、兼容 Vue 架构、易于编写维护测试脚本
单元测试	xUnit (ASP.NET Core)	后端业务逻辑层单元测试	与 ASP.NET Core 无缝集成，支持 Mock、断言与代码覆盖率
性能测试	JMeter	高并发模拟、接口压测	支持参数化、图形展示、脚本复用，适配多种协议与格式

6.2 手工测试辅助工具

工具名称	用途	理由说明
Fiddler	抓包与网络请求分析	辅助排查接口调用问题，验证前后端数据一致性
Oracle SQL Developer	数据库验证与测试数据构造	提供图形化界面管理 Oracle 数据库，支持复杂查询与导出
Trello / 禅道	缺陷记录与测试流程管理	支持测试用例跟踪、缺陷生命周期管理、任务协作

6.3 框架选择依据

1. **技术适配性**：后端基于 ASP.NET Core，使用 xUnit 可与项目程序集成测试；前端基于 Vue.js，Selenium 可有效模拟页面交互逻辑。
2. **团队熟悉度**：大部分测试成员熟悉 Postman、Selenium 与 JMeter 的使用，可降低学习成本、提升执行效率。
3. **测试目标匹配**：上述框架分别覆盖了单元测试、接口测试、前端自动化测试与性能测试四大核心维度，满足平台测试要求。
4. **可维护性与扩展性**：支持持续集成（CI）环境集成与自动化回归能力，可随版本迭代快速验证核心功能稳定性。
5. **开源免费**：所选框架均为开源或免费使用，便于学生团队项目使用，无额外预算压力。

7 成本估算

7.1 资源需求

- 测试人员：6人
- 测试工具：自动化测试工具，包括Selenium WebDriver、JUnit、WebMvc、APIFox等
- 测试环境：基于ASP.NET Core和Vue框架的开发环境及测试环境，软件开发IDE为VScode

7.2 工作量估算

根据项目甘特图安排，测试阶段共需6名测试人员投入18个工作日，总工作量为6人 × 18天 = 108人天。

7.3 成本计算

- 人力成本：
假设6名测试人员的日均薪资一致，均为180元/人/天，
则总人力成本为108人天 × 180元/人天 = 19,440元。
- 工具成本：
所使用的测试工具均为免费工具，无额外采购费用。
- 环境成本：
测试与开发环境均为现有环境，无需额外投入。

综上，本次测试阶段的总成本为19,440元。

7.4 相关因素及影响

- 代码质量和可测试性对测试成本具有重要影响。
- 低质量或测试友好性差的代码，可能导致自动化测试脚本的编写与执行时间增加，从而增加人力资源投入和整体成本。
- 若在软件开发的早期阶段注重代码规范和提升测试性，将有效减少测试工作量，降低后续维护成本。
- 因此，提升代码质量及测试设计的合理性，是优化测试成本的关键举措。